ILDS - Intelligent Leak Detection System

Complete Cloud Architecture Guide

Understand the entire system in one reading

🎯 THE BIG PICTURE

ILDS is a 24/7 AI guard for oil/gas pipelines.

It uses **two pressure sensors** (BFA8 at Rewari, BFA3 at Asadpur Khera) to **detect leaks in real-time** by analyzing pressure waves traveling at **1,090 m/s** along a **37.28 km pipeline**.

Cloud Stack:
`S3 (Storage) → SNS (Event Broadcaster) → SQS (Message Queue) → EC2 (Brain) → SNS (Alert System)`

🔄 THE 7-STAGE DATA FLOW

📥 STAGE 1: DATA INGESTION (The Entry Gate)

What happens? Sensor CSV files arrive in S3.

Flow: `Sensor (100Hz) → CSV File → S3 Bucket → S3 Event → SNS Topic → SQS Queue → EC2 Polling`

Key Rules (Axioms):

1. **Only real-time files** (≤ 90 minutes old) are processed. Older files = ignored.
2. **Long-polling magic**: SQS waits **20 seconds** for messages (no CPU waste).
3. **File naming**: `BFA8_Batch{N}_{YYYY-MM-DD}_{HH-MM-SS}.csv` (contains voltage data).
4. **Local cache**: EC2 keeps **last 20 files per sensor** for fast access (`/home/ubuntu/bfa/data/BFA8/`).
5. **Dashboard heartbeat**: Every file upload updates status to **ACTIVE** (goes **OFF** if no update in 15 mins).

Why? Ensures only fresh, relevant data enters the system.

🔍 STAGE 2: PAIR MATCHING (The Twin Finder)

What happens? Find matching files from **both sensors** within a **5-minute window**.

Flow: `EC2 checks: Do I have BFA8 + BFA3 files with similar timestamps?`

Key Rules:

1. **Temporal match**: `|time_BFA8 - time_BFA3| ≤ 5 minutes`.

2. **Fallback to S3**: If local files missing, download top-3 candidates from S3.
3. **No duplicates**: Checks `processed_pairs.csv` to skip already-processed pairs.
4. **No match?** Delete SQS message, wait for next trigger.

Why? Leak detection needs **synchronized data** from both ends of the pipeline.

⚙️ STAGE 3: BATCH PROCESSING (The Data Chef)

What happens? Clean, align, and standardize the data for analysis.

Flow: `Raw CSV → Voltage→Pressure → Filter → Interpolate → Save Aligned Batch`

Key Rules:

1. **Minimum overlap**: **6 minutes (36,000 samples @ 100Hz)** required.
   - If overlap < 6 mins → **backtrack** to older files until requirement met.
2. **Voltage → Pressure**:
   - Formula: `Pressure = 1.0818 × Voltage - 0.0086`
   - Range: **0.72V–3.60V → 0–100 BAR**
3. **Signal filtering**:
   - Window: **1,000 samples**
   - Steps: Pad → Median → Moving Average → Trim
4. **Interpolation**:
   - Uniform grid of **36,000 samples** (6 mins @ 100Hz).
   - Timestamps converted from **UTC → IST (Asia/Kolkata)**.
5. **Output**:
   - `BFA8_concat_{timestamp}.csv`
   - `BFA3_concat_{timestamp}.csv`
   - Columns: **Timestamp (IST) + Pressure (BAR)**

Why? Raw data is noisy; this stage makes it **analysis-ready**.

⚠️ STAGE 4: HPC CHECK (The False Alarm Killer)

What happens? Detect **High Pressure Changes (HPC)** to avoid false leak alerts.

Flow: `Is |max(Pressure) - min(Pressure)| > threshold?`

Key Rules:

1. **Thresholds**:
   - **Rewari (BFA8)**: > **3.0 BAR** (valve/pig/surge/shutdown events)
   - **Asadpur (BFA3)**: > **0.25 BAR**
2. **If HPC detected**:
   - Return **NORMAL** immediately (not a leak, just an operational event).
   - Log: `"OPERATIONAL Change detected"`.

3. **HPC Revert Logic**:
    - If **previous classification was HPC-NORMAL** and current is LEAK → **revert LEAK to NORMAL**.
    - Prevents false positives after operational events.

Why? Leaks and operational events both cause pressure changes—**HPC filters out the non-leaks**.

🚦 STAGE 5: FLOW CLASSIFICATION (Static vs. Dynamic)

What happens? Determine if the pipeline is **flowing or static**.

Flow: `Pressure Difference (ΔP) = |mean(BFA8) - mean(BFA3)|`

Key Rules:

1. **Static Flow**: ΔP **< 0.75 BAR** (near-zero flow or shutdown).
2. **Dynamic Flow**: ΔP **≥ 0.75 BAR** (normal operation).
3. **Flow rate formula** (for logging): `Flowrate = 345.6098 + 280.3044 × √(ΔP)`

Why? Leak detection logic differs for **static vs. dynamic** conditions.

🎯 STAGE 6: LOGIC8 DETECTION (The Leak Hunter)

What happens? Find **pressure troughs** (dips) that indicate leaks.

Flow: `Detrend → Find Peaks → Match Troughs → Calculate Location → Estimate Size`

Key Rules:

1. **Sliding window**:
    - Window: **3 minutes**
    - Shift: **1 minute**
    - Sample rate: **100Hz**
2. **Detrend**: Remove linear trends to isolate **transient events** (leaks).
3. **Peak detection (2-pass)**:
    - **Pass 1**: Prominence thresholds (BFA8: 0.02–0.5 BAR, BFA3: 0.02–0.3 BAR).
    - **Pass 2**: If no peaks, **reduce thresholds** and retry.
4. **Trough matching**:
    - Match troughs from **both sensors** within **[-5, 30] seconds**.
    - If no match → **NORMAL**.
5. **Location calculation**:
    - **Method 1 (Trough Times)**: `Distance = (Pipeline_Length + WaveSpeed × Δt) / 2`
        - Pipeline length: **37,280m**
        - Wave speed: **1,090 m/s**
    - **Method 2 (Cross-correlation)**: More precise, uses signal correlation.
6. **Validation**:
    - Location must be **2,000m–35,000m** (valid pipeline segment).
    - If out of range → **NORMAL**.

7. **Leak size estimation**:
   - Formula: `Leak % = 0.00628 × exp(1.58 × Prominence)`
   - Clipped to **0–5%** range.
   - If < 0.5% → `"<0.5%"`; else `">0.5%"`.
8. **Consecutive leak correction**:
   - If a leak was detected in the **last 15 minutes**, adjust location to **14,710m ± 200m** (reference point).

Why? Leaks create **characteristic pressure waves** that travel at known speeds—this stage **pinpoints the leak**.

🚨 STAGE 7: FINAL CLASSIFICATION & ALERT (The Decision Maker)

What happens? Decide: **LEAK or NORMAL**, then alert if needed.

Flow: `Merge all paths → Classify → Log → Alert → Update Dashboard`

Key Rules:

1. **Final classification**:
   - If **LEAK**: `cls = "LEAK"`, `location_m`, `leak_size`, `p1_time`.
   - If **NORMAL**: `cls = "NORMAL"`.
2. **Logging**:
   - **Main results**: `leak_detection_results_v2.csv`
   - **Logic8 details**: `leak_detection_results_logic8.csv`
   - **Dashboard updates**: `delta_t_results.json`, `dashboard_updates.json`
3. **Alerting**:
   - **SNS Topic**: `ilds_alerts`
   - **Message**: `ILDS Alert: LEAK DETECTED | Status: LEAK | Location: {m} | Timestamp: {IST} | Leak Size: {+}%`
4. **Dashboard update**:
   - Status: **LEAK/NORMAL**
   - Location: **km from Rewari**
   - Leak size: **%**
   - Timestamp: **IST**

Why? This is where the system **takes action**—sending alerts to operators.

🧠 7 CORE AXIOMS (Memorize These)

1. 🔹 **The 90-Minute Rule**: Only files **≤ 90 mins old** are processed (real-time focus).
2. 🔹 **The 5-Minute Window**: Sensor files must match within **5 minutes** to be paired.
3. 🔹 **The 6-Minute Minimum**: Need **36,000 samples (6 mins @ 100Hz)** for analysis.
4. 🔹 **The 30-Second Window**: Troughs must align within **[-5, 30] seconds** to be matched.
5. 🔹 **The HPC Thresholds**:
   - Rewari: **> 3.0 BAR** = operational event (not a leak).
   - Asadpur: **> 0.25 BAR**.

6. 🔹 **The Static/Dynamic Split**: **< 0.75 BAR ΔP = Static, ≥ 0.75 BAR = Dynamic**.
7. 🔹 **The Pipeline Constants**:
   - Length: **37,280m**
   - Wave speed: **1,090 m/s**
   - Valid leak range: **2,000m–35,000m**

🏗️ ARCHITECTURE PRINCIPLES

Principle	How It's Applied	Benefit
Event-Driven	S3 uploads trigger SNS → SQS → EC2	No polling waste, real-time
Decoupled	S3 → SNS → SQS separates storage, events, and processing	Scalable, resilient
Stateless	EC2 processes messages; state stored in S3/SQS	Easy to scale, recover
Idempotent	`processed_pairs.csv` prevents duplicate processing	No double-work
Self-Correcting	HPC revert logic + consecutive leak correction	Fewer false positives
Redundant	Two location calculation methods (trough times + cross-correlation)	More accurate
Real-Time	90-minute cutoff + long-polling	Focus on live data

🎭 MENTAL MODEL (The Story to Remember)

Imagine **ILDS as a security guard** patrolling a **37.28 km pipeline**:

1. 👀 **Eyes (Sensors)**: BFA8 (Rewari) and BFA3 (Asadpur Khera) **watch pressure 100 times per second**.
2. 📝 **Notebook (S3)**: Every observation (CSV file) is **logged in a notebook**.
3. 📢 **Radio (SNS)**: The notebook **broadcasts** new entries to the guard's radio.
4. 📥 **Inbox (SQS)**: The guard's **inbox** collects radio messages (waits 20 secs for new ones).
5. 🧠 **Brain (EC2)**: The guard **reads the inbox**, checks:
   - *"Are these observations from both ends?"* (Pair Matching)
   - *"Are they recent?"* (90-minute rule)
   - *"Is there enough overlap?"* (6-minute rule)
   - *"Is this just a valve opening?"* (HPC Check)
   - *"Is the pipeline flowing or static?"* (Flow Classification)
   - *"Do I see a leak's pressure wave?"* (Logic8 Detection)
6. 🚨 **Alarm (SNS)**: If a **leak is confirmed**, the guard **sounds the alarm** with location and size.
7. 📊 **Dashboard**: The guard **updates a status board** (Dashboard) for operators to see.

🔍 QUICK REFERENCE CHEAT SHEET

Component	Purpose	Key Details
S3 Bucket	Store raw CSV files	`471112814693-bfa-dev-ap-south-1-ilds-transmitter-data`
SNS Topic	Broadcast S3 events	Wraps S3 events for SQS
SQS Queue	Message queue	`ilds_queue_1`, long-poll: 20s, max 10 msgs

Component	Purpose	Key Details
EC2 Service	Main processing	Runs `run_service.py`, 24/7, cycle-based
Local Cache	Fast file access	`/home/ubuntu/bfa/data/BFA8/`, last 20 files
HPC Check	Filter false alarms	Thresholds: 3.0 BAR (Rewari), 0.25 BAR (Asadpur)
Logic8	Leak detection	Sliding window, peak detection, location calculation
SNS Alerts	Notify operators	Topic: `ilds_alerts`, includes location + size
Dashboard	Visual status	`dashboard_updates.json`, `delta_t_results.json`

✅ ONE-SENTENCE SUMMARY PER STAGE

1. **Ingestion**: Sensors upload CSV files to S3, which triggers SNS→SQS→EC2.
2. **Pairing**: EC2 matches BFA8 + BFA3 files within 5 minutes.
3. **Processing**: Data is cleaned, aligned, and interpolated to 36,000 samples.
4. **HPC Check**: High pressure changes (operational events) are filtered out.
5. **Flow Classification**: Pipeline is classified as **static** (<0.75 BAR ΔP) or **dynamic** (≥0.75 BAR).
6. **Leak Detection**: Logic8 finds pressure troughs, matches them, and calculates **location + size**.
7. **Alerting**: If LEAK → **SNS alert** + **dashboard update**; if NORMAL → **log only**.

💡 KEY NUMBERS TO REMEMBER

Concept	Value	Purpose
File Age Cutoff	**90 minutes**	Real-time focus
Pair Matching Window	**5 minutes**	Synchronize sensors
Minimum Overlap	**6 minutes (36,000 samples)**	Data sufficiency
Trough Matching Window	**[-5, 30] seconds**	Align pressure events
Rewari HPC Threshold	**> 3.0 BAR**	Filter operational events
Asadpur HPC Threshold	**> 0.25 BAR**	Filter operational events
Static/Dynamic Split	**0.75 BAR ΔP**	Flow classification
Pipeline Length	**37,280 meters**	Location calculation
Wave Speed	**1,090 m/s**	Pressure wave propagation
Valid Leak Range	**2,000m–35,000m**	Pipeline segment

📚 GLOSSARY

Term	Definition
BFA8	Sensor at Rewari station
BFA3	Sensor at Asadpur Khera RCP
HPC	High Pressure Change (operational event, not a leak)

Term	Definition
Logic8	Leak detection algorithm using pressure trough analysis
IST	Indian Standard Time (UTC+5:30)
SQS Long-Polling	Waits up to 20 seconds for messages (reduces API calls)
Prominence	Height of a peak relative to surrounding points
Cross-correlation	Statistical method to find time lag between signals

---

🎯 FINAL TAKEAWAY

You now understand ILDS completely.

The system is designed with **7 clear stages**, each with **simple, memorable rules**. The architecture is **event-driven, decoupled, and self-correcting**, ensuring reliable leak detection with minimal false positives.

Remember the mental model: A security guard patrolling a pipeline, using sensors as eyes, S3 as a notebook, and EC2 as the brain to detect and alert on leaks.

Document generated from ILDS flowchart analysis | Last updated: August 25, 2026